P.Vishwa Nandan(192473015)

Q5 – Derive a Generic Ranking Component from a Principle

Marks: 7

1. Introduction

A sports leaderboard is used to rank teams based on their performance.

The ranking follows three conditions:

1. Teams with higher points receive higher ranks.
2. If points are equal, the team with a higher goal difference receives a higher rank.
3. If both points and goal difference are equal, team names are arranged alphabetically.

The same ranking component can also be reused for ranking players, colleges and quiz teams.

2. Comparable and Comparator

Java provides two interfaces for comparing objects.

A. Comparable

Comparable is used when a class defines its own natural ordering.

It uses the compareTo() method.

Example:

```
class Team implements Comparable<Team> {

   String name;
   int points;

   @Override
   public int compareTo(Team other) {
      return Integer.compare(other.points, this.points);
   }
}
```

Here, teams are compared based on points.

Advantages:

- Defines natural ordering inside the class.

- Useful when a class has one standard ordering.

Disadvantage:

- Different ranking rules are difficult to maintain within one natural ordering.

B. Comparator

Comparator is used when comparison rules are defined separately from the class.

It uses the compare() method.

Example:

```
Comparator<Team> byPoints =
   Comparator.comparingInt((Team t) -> t.points).reversed();
```

Advantages:

1. Different comparison rules can be created.
2. The original class does not need modification.
3. Multiple ranking methods can be used.
4. It supports reusable ranking applications.

Comparison

Comparable	**Comparator**
Uses compareTo()	Uses compare()
Defined inside the class	Defined outside the class
Provides natural ordering	Provides custom ordering
Usually one natural order	Supports multiple ordering rules

Selection: Comparator is preferred because the same leaderboard component must support different ranking rules for teams, players, colleges and quiz teams.

3. Multi-Field Comparison Using Comparator

The ranking uses three conditions.

First: Points

Higher points must come first.

```
Comparator.comparingInt((Team t) -> t.points).reversed()
```

Second: Goal Difference

If points are equal, higher goal difference must come first.

```
.thenComparing(
    Comparator.comparingInt((Team t) -> t.goalDiff).reversed()
)
```

Third: Team Name

If both points and goal difference are equal, team names are arranged alphabetically.

```
.thenComparing(t -> t.name)
```

Complete Comparator

```
Comparator<Team> byRank =
    Comparator.comparingInt((Team t) -> t.points).reversed()
    .thenComparing(
        Comparator.comparingInt((Team t) -> t.goalDiff).reversed()
    )
    .thenComparing(t -> t.name);
```

The reversed() method arranges points and goal difference in descending order.

The final condition arranges team names in ascending alphabetical order.

4. Generic Class – Leaderboard<T>

Generics allow a class to work with different data types.

For example:

- Leaderboard<Team>
- Leaderboard<Player>
- Leaderboard<College>
- Leaderboard<QuizTeam>

Instead of creating separate classes for each type, we create one reusable generic class.

Example:

```
class Leaderboard<T> {

    private final List<T> items = new ArrayList<>();
```

```
    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }

    void add(T item) {
        items.add(item);
    }
}
```

Here, T represents the type of object stored in the leaderboard.

A Comparator is supplied to define how the objects should be ranked.

Bounded Types

Since the Comparator is supplied separately, the generic Leaderboard class does not require a specific type bound.

If Comparable were used instead, the declaration could be:

class Leaderboard<T extends Comparable<? super T>>

This ensures that T supports natural ordering.

5. Use of Wildcard – List<? extends T>

Wildcards allow generic code to work with related types.

List<? extends T> is an upper-bounded wildcard.

It means the list can contain objects of type T or its subclasses.

Example:

```
void addAll(List<? extends T> newItems) {
    items.addAll(newItems);
}
```

This method allows the leaderboard to accept lists containing different subclasses of T.

It improves flexibility and reusability when working with related object types.

6. Complete the Code – Generic Leaderboard

The assignment provides two incomplete sections.

- TODO 1: Return a new sorted copy of the items without modifying the original list.
- TODO 2: Create the comparator using points, goal difference and team name.

Java Program

File name: Main.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Team {

    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override
    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
```

```
    private final List<T> items = new ArrayList<>();

    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }

    // Add an item to the leaderboard
    void add(T item) {
        items.add(item);
    }

    // Accept a list of T or its subclasses
    void addAll(List<? extends T> newItems) {
        items.addAll(newItems);
    }

    // TODO 1: Return a new sorted copy without modifying items
    List<T> ranking() {

        List<T> sortedItems = new ArrayList<>(items);

        sortedItems.sort(order);

        return sortedItems;
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        // TODO 2: Build the comparator
        Comparator<Team> byRank =
                Comparator.comparingInt(
                        (Team t) -> t.points
                ).reversed()

                .thenComparing(
                        Comparator.comparingInt(
                                (Team t) -> t.goalDiff
                        ).reversed()
                )

                .thenComparing(t -> t.name);

        // Create leaderboard
        Leaderboard<Team> board =
                new Leaderboard<>(byRank);

        // Add teams
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));

        // Display ranking
```

```
        System.out.println(board.ranking());
    }
}
```

7. Expected Output

[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]

Explanation of Output

First – Eagles

Points = 9, Goal Difference = 6.

Eagles receive first place because they have the highest goal difference among teams with 9 points.

Second – Bisons

Points = 9, Goal Difference = 4.

Third – Panthers

Points = 9, Goal Difference = 4.

Bisons are ranked above Panthers because both have equal points and goal difference, but Bisons comes alphabetically before Panthers.

Fourth – Cobras

Points = 6, Goal Difference = 1.

Cobras receive fourth place because they have fewer points.

8. Additional Testing – Faulty Comparator

The assignment requires a second dataset designed to expose an incorrect comparator.

Consider the following teams:

Team	Points	Goal Difference
Alpha	10	3
Gamma	10	8
Beta	10	8
Delta	8	9

Faulty Comparator

Suppose we compare only points and ignore goal difference and team name.

```
Comparator<Team> faultyComparator =
    Comparator.comparingInt((Team t) -> t.points).reversed();
```

This comparator does not implement all three ranking conditions.

Faulty Output

Since Java sorting is stable, teams with equal points retain their original relative order.

[Alpha(10,3), Gamma(10,8), Beta(10,8), Delta(8,9)]

Correct Output

[Beta(10,8), Gamma(10,8), Alpha(10,3), Delta(8,9)]

The correct output ranks Beta above Gamma alphabetically because they have equal points and goal difference.

Alpha comes after them because its goal difference is lower.

The faulty comparator produces an incorrect ranking because it ignores the second and third conditions.

We should also avoid comparing integers using subtraction, such as other.points - this.points, because subtraction can overflow. Comparator.comparingInt() and Integer.compare() are safer.

9. Correctness of the Comparator

The comparator follows the three ranking conditions in order.

1. Higher points receive higher ranks.
2. If points are equal, higher goal difference receives higher ranks.
3. If both are equal, alphabetical order is applied.

The comparison is performed lexicographically. The next condition is considered only when the previous condition produces equality.

This gives a consistent ordering.

If all three fields are equal, the comparator returns zero.

Therefore, the comparator satisfies the ranking principle given in the assignment.

10. Alternative Concept and Complexity Analysis

The assignment asks us to compare Comparator with Comparable and three stable sorting operations.

A. Using Comparable<Team>

Instead of using Comparator, Team can implement Comparable.

Example:

```
class Team implements Comparable<Team> {

    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override
    public int compareTo(Team other) {

        int result =
                Integer.compare(other.points, this.points);

        if (result != 0) {
            return result;
        }

        result =
                Integer.compare(other.goalDiff, this.goalDiff);

        if (result != 0) {
            return result;
        }
```

```
        return this.name.compareTo(other.name);
    }
}
```

Here, compareTo() defines the natural ranking order inside the Team class.

Advantages:

- Defines natural ordering inside the class.
- Sorting can be performed using Collections.sort().
- Suitable when only one standard ranking is required.

Disadvantage:

- Different ranking rules require additional Comparator implementations.

B. Three Stable Sorting Operations

Another approach is to sort the list three times.

The sorting order starts with the least important field.

1. First sort by team name.
2. Then sort by goal difference in descending order.
3. Finally sort by points in descending order.

Example:

```
teams.sort(Comparator.comparing(t -> t.name));


teams.sort(
    Comparator.comparingInt((Team t) -> t.goalDiff).reversed()
);


teams.sort(
    Comparator.comparingInt((Team t) -> t.points).reversed()
);
```

Java sorting is stable, meaning elements with equal values retain their previous relative order.

Therefore, the earlier sorting priorities are preserved.

C. Complexity Analysis

Let n represent the number of teams.

1. Comparator Chain

- Time complexity: O(n log n).
- Extra space: O(n) for the sorted copy and sorting workspace.
- Requires one sorting operation.

2. Comparable

- Time complexity: O(n log n).
- Extra space: O(n) for sorting workspace.
- Uses natural ordering defined inside the Team class.

3. Three Stable Sorts

- First sorting operation: O(n log n).
- Second sorting operation: O(n log n).
- Third sorting operation: O(n log n).
- Total: O(3n log n), which is asymptotically O(n log n).
- Extra space: O(n) for sorting workspace.

Although all approaches have the same Big-O time complexity, three sorting operations perform more sorting passes.

D. Comparison

Feature	**Comparator Chain**	**Comparable**	**Three Stable Sorts**
Sorting operations	1	1	3
Time complexity	O(n log n)	O(n log n)	O(n log n)
Extra space	O(n)	O(n)	O(n)
Flexible ranking rules	Yes	Limited	Yes
Reusability	High	Moderate	Moderate
Maintenance	Easy	Depends on class	More operations

Recommendation

Comparator is preferred because it separates ranking rules from the Team class.

The same generic Leaderboard component can be reused for players, colleges and quiz teams by supplying different Comparator objects.

11. Conclusion

Comparator provides flexible and reusable object comparison in Java. The generic Leaderboard class works with different object types using generics. Comparator chaining ranks teams based on points, goal difference and alphabetical order. Wildcards improve flexibility when handling related types. Therefore, Comparator is preferred over a fixed natural ordering because the same ranking component can be reused for different applications.